

Retry Synthesis with Bounded Calls and Protected Work

ANONYMOUS AUTHOR(S)

We synthesize retry policies for dependent transformations using cheap validation, fresh protected computation, and cached reuse or recomputation. One policy, given neither writer budget nor prices, permits r extra calls and simultaneously minimizes worst protected work for every writer budget B : the sum of the largest $\min((B - r)_+, n)$ job works. Without cached completion, all work must be protected already at $B = r$. A potential proves a normal form over retained preparations; Java identity callbacks admit matching adversaries with actual writes. Minimum-work ready selection is optimal within the threshold family; an exact suffix certificate permits further protection choices without weakening either resource guarantee. Scalar all-budget values have two slopes per job and $2^n \text{poly}(L)$ construction/checking; a mixed-price class reduces this to $O(|E| + n \log(n + 1))$. Native counters and certificates test these guarantees. A Roslyn source modification bounds update calls over 4,019 repository-derived documents, while a controlled Java probe demonstrates reduced collateral blocking. Roslyn timings mostly do not improve, and API calibration finds no measured third-mode benefit. These are counted-resource guarantees, not general application speedups.

CCS Concepts: • **Software and its engineering** → **Formal software verification**; *Concurrent programming structures*.

Additional Key Words and Phrases: program synthesis, concurrency, retries, quantitative games, resource guarantees

1 Introduction

A library may cap retries without knowing how often concurrent writers will intervene. Computing under a lock after this cap ensures completion, but can block unrelated work using the same serialization point. We ask which transformations must run under protection, and how to synthesize retries, fresh protected computation and cached completion.

For dependent kernels of 8 and 16 selected operations, let W, L count total/protected work and Q completion calls. With no extra calls, cached completion gives $Q = 2$ and worst $L = 0, 16, 24$ for at most $B = 0, 1, 2$ writes. Every budget-unaware two-mode program with the same call cap must protect all 24 operations even when quiet (Theorem 4). Cached completion changes joint resource feasibility without a price or writer-bound input.

Roslyn’s document-update loop [2] provides a source case without a supplied writer bound: a capped variant preserves completed immutable solutions while limiting serialization calls. Its transformations have variable/lazy costs, so only the call/transformation counts transfer. A separate Deephaven case imports bounded-work profiles under caller-owned refreshes and exposes abstraction slack.

Optimistic fallback [1, 19], budgeted DP and input-slope envelopes [9, 28, 32] are established. The contribution is their justified specialization to charged atomic calls: the normal-form potential, simultaneous resource frontier and exact suffix feasibility without a writer-budget input, finite slope-basis closure, sharp independent-job ratio, and exact charged-price reduction that removes both budget and subset enumeration on a stated class. Authored experiments test these claims and their implementation.

2 Charged Atomic Calls

Write $\mathbb{N} = \{0, 1, \dots\}$. Jobs $J = \{1, \dots, n\}$ form a finite DAG with predecessor set $\text{Pred}(i)$. Implementation IDs start at zero; readiness requires all predecessors completed. Completion records the exact immutable output and persists despite later external writes. The objective is one successful foreground transformation per job, not a simultaneous snapshot or transaction.

Each ready job has a pure, opaque, whole kernel receiving its own-input identity and exact completed predecessor outputs. Its fixed unprotected work is $w_i > 0$; under protection it costs $w_i + p_i$, $p_i \geq 0$, including predecessor processing. Kernels finish with exact results; partial/incremental evaluation, preparation before readiness and discharge by external work are excluded. A preparation is a capability tied to its input identity and predecessor tuple. Captured outputs cannot be reconstructed by later reads of the mutable map.

A local state is U (no usable current preparation), C (a usable current preparation), or D (completed). Several ready jobs may simultaneously be in C. Initially every job is U. An invalidation changes an own-input identity, preserving captured predecessor outputs. The environment has a global remaining allowance b , initially $B \in \mathbb{N}$. At zero allowance only matching outcomes are legal.

The declared charge $v_i \geq 0$ covers an identity-conditional publication call. The charge $k_i \geq 0$ covers entry and exit of an atomic callback, including a callback that only validates. It may be supplied as $g_i + r_i$. These charges define a resource metric; they are not measurements of API latency. All prices are integers, or rational prices scaled uniformly to integers. Table 1 defines the available counted operations. A callback holds protection only during that operation and performs no other job's foreground work.

Table 1. Counted atomic-call interface. A mismatch spends one invalidation token. Cached completion always finishes the job; validation-only mismatch retains the current live entry and reports failure.

Operation	Start	Matching outcome/cost	Mismatching outcome/cost
Prepare	U	C, w_i	—
Conditional publication	C	D, v_i	U, v_i
Validation-only callback	C	D, k_i	U, k_i
Fresh callback	U	D, $k_i + w_i + p_i$	—
Cached callback	C	D, k_i	D, $k_i + w_i + p_i$

Discarding a preparation is free. A free inspection is permitted only with a legal outcome that preserves a currently usable preparation; it cannot certify that future interference is exhausted. Programs can select any ready job after any completed call, retain preparations or repeat inspections. There is no bare acquire/release operation, guard-retaining return or cross-job batch callback. All calls finish, and every noncompleting program, including one executing free cycles forever, has infinite objective. The environment resolves outcomes after observing the program's action.

Exposed, retained guards permit choices unavailable inside this call interface. Positive guard charges defeat some serial-macro arguments; the distinct zero-fee result and charged counterexamples remain in Section 10.

3 A Three-Mode Normal Form

Write S for the unfinished jobs and

$$\mathcal{A}(S) = \{i \in S : \text{Pred}(i) \cap S = \emptyset\}$$

for available jobs. Reachable unfinished sets are upsets. For proofs we extend the game to all subsets, treating absent predecessors as completed. Define

$$\begin{aligned} m_i &= \min(v_i, k_i), & c_i &= w_i + m_i, \\ \delta_i &= k_i - m_i, & P_i &= p_i + \delta_i, \\ s_i &= w_i + p_i. \end{aligned} \tag{1}$$

An unavoidable baseline from an all-unprepared start is $A(S) = \sum_{i \in S} c_i$. Three serial modes remain at boundaries where unfinished jobs have no retained preparation:

- (1) *Protected*: execute a fresh callback, paying $c_i + P_i$, and remove i .
- (2) *Cheap validation*: prepare and invoke the cheaper validation API. A match pays c_i and removes i ; a mismatch pays c_i , leaves i unprepared and spends one token.
- (3) *Cached completion*: prepare and invoke a cached callback. A match pays $c_i + \delta_i$; a mismatch pays $c_i + \delta_i + s_i$ and spends one token. Both remove i .

Subtract the once-paid baseline. The optimal worst excess of these modes satisfies $F(\emptyset, b) = F(S, 0) = 0$ and, for $b > 0$,

$$F(S, b) = \min_{i \in \mathcal{A}(S)} \left\{ \begin{array}{l} P_i + F(S - i, b), \\ \max\{F(S - i, b), c_i + F(S, b - 1)\}, \\ \delta_i + \max\{F(S - i, b), s_i + F(S - i, b - 1)\} \end{array} \right\}, \quad (2)$$

where $S - i = S \setminus \{i\}$. At $b = 0$ cheap validation attains zero excess; the cached option has excess δ_i , without a mismatch branch. Each mode outcome decreases $|S| + b$, so a minimizing deterministic controller finishes within $n + B$ mode invocations. This is worst-path control against an adaptive environment, not expected-cost control against a fixed distribution.

THEOREM 1 (CHARGED ATOMIC-CALL NORMAL FORM). *For the interface of Section 2, the minimum worst cost over all causal programs from the all-unprepared state is*

$$\text{OPT}(J, B) = \sum_{i \in J} c_i + F(J, B).$$

A serial three-mode controller attains it. Programs retaining several preparations or changing jobs between calls cannot improve the value.

PROOF. The three modes implement Table 1 with exactly Equation (2)'s full costs. Their decreasing rank and captured successful outputs give the upper bound.

For the lower bound, let x be any interface state with unfinished set T . Define

$$\Phi(x, b) = \sum_{i: x_i = \text{U}} c_i + \sum_{i: x_i = \text{C}} m_i + F(T, b). \quad (3)$$

Cached jobs remain ready because completion persists. For each operation we show a legal outcome preserving immediate cost plus successor potential. Preparation pays $w_i = c_i - m_i$ and replaces c_i by m_i . Validation with charge $z_i \in \{v_i, k_i\}$ gives excess $z_i - m_i + F(T - i, b)$ on success or $w_i + z_i + F(T, b - 1)$ on failure. Since $z_i \geq m_i$, their maximum dominates the cheap branch; at $b = 0$ success suffices. Fresh callbacks use the protected inequality. Cached callbacks give excesses $\delta_i + F(T - i, b)$ and $\delta_i + s_i + F(T - i, b - 1)$, exactly the cached branch. Discarding C raises potential by w_i ; inspection admits a preserving outcome.

Nature selects a witnessing branch after each action, within budget b . Telescoping gives the initial potential as a cost lower bound, covering arbitrary job switching and retained preparations. Noncompletion costs infinity; intermediate potentials need not equal intermediate optima. $\square$

A resource cap M is feasible exactly when $A(J) + F(J, B) \leq M$. Prices and allowance are supplied; restricted or correlated outcomes preserve the upper guarantee but may lower the optimum.

4 Actual External Writes in Java

Repeated stale rejection need not require a new write; writes to completed parents preserve captured outputs.

Use Java 17’s `ConcurrentHashMap` with one foreground thread, distinct persistent keys, immutable identity-equal wrappers and fresh, never-reused identities for external writes. Each job updates only its fixed key. Keys are not removed; results are non-null; kernels use own snapshots and captured parents and terminate without exceptions. Conditional replacement accepts identity or equals equality [25, 26]; atomic compute invokes its callback once, under a bin monitor in the inspected source [24, 26]. Different keys may share bins.

Implement Table 1 using `replace(key, expected, result)` and `compute`. Validation-only callbacks return the prepared result on match, otherwise the unchanged entry with a failure flag; cached callbacks recompute on mismatch, and fresh callbacks always compute under protection. Capture permanent milestones locally rather than rereading the map.

Callbacks perform no other-key map operations, other foreground kernels or direct guard work, including on cached matches; no guard survives. Multiple preparations are allowed. Prices charge the specified calls; bin collisions add no unperformed kernel work. Assume scheduling and call completion: finite write counts do not imply fairness.

THEOREM 2 (MATCHING ACTUAL-WRITE ENVIRONMENT). *Under these conditions, with no initial preparations and at most B successful external writes,*

$$\min_P \sup_{E: \# \text{writes}(E) \leq B} \text{Cost}(P, E) = \sum_i c_i + F(J, B).$$

The equality holds for separate or shared internal bins.

PROOF. Classify job i as C exactly when some retained preparation has its current identity, otherwise U. Use Equation (3) with $b = B - \# \text{writes}$. Immediately before a comparison acquires its monitor, the environment can preserve a current preparation or invalidate it with one fresh own-key write. Theorem 1’s validation/cached inequalities apply. No monitor survives the boundary, so this works with colliding bins.

Fresh preparation performs the U-to-C credit transfer; computing from an old identity adds no current capability. Stale validation rejects without a write and leaves the semantic state unchanged. A stale cached callback recomputes inside: its excess is $k_i + s_i - c_i = P_i$ in state U, or $k_i + s_i - m_i = w_i + P_i$ if another current preparation exists. Both dominate the protected option. Fresh callbacks use that option directly. Discarding the only current preparation increases potential; copies, stale preparations and preserving inspections cannot decrease it. Captured parents survive live-parent writes. Thus every operation admits a zero/one-write continuation preserving cost plus potential; telescoping gives the lower bound with at most B writes. The interface forbids using a shared bin to perform another job’s work inside the same call.

For the upper bound, the serial controller freshly reads and prepares after every cheap failure. Its read-to-comparison intervals, including cached completions, are disjoint; each mismatch contains a distinct write. There are at most B mismatches. Counting them may overestimate remaining writes, conservatively, so the three-mode guarantee applies in either layout. $\square$

Operational accounting. Native kernels fold captured predecessor seeds and an immutable array of length $8w_i - |\text{Pred}(i)| > 0$, executing exactly $8w_i$ selected operations. The metric adds $8p_i$ per protected kernel, $8v_i$ per conditional call and $8k_i$ per callback, including mismatches. It scales the abstract objective by eight. Allocation, monitor waiting, initialization, background work and other API instructions are excluded; charges are not inferred from timing.

4.1 A Protected-Work Frontier Without Fitted Prices

Let W count all foreground kernel work, L the part performed inside atomic callbacks, and Q all foreground completion/validation calls, including failed conditionals. Reads, initialization and external-writer calls are excluded from Q . Here w_i is each complete kernel's mandatory work on every input, inside or outside protection, not merely an upper bound. Let $\Omega_k(T)$ sum the largest $\min(k, |T|)$ works in T , and $\Omega(T)$ its total. Omitted sets mean J ; $\Omega_0 = 0$ and $\Omega_n = \Omega$.

THEOREM 3 (PROTECTED WORK UNDER A CALL CAP). *Under Theorem 2's interface and environment, fix integers $B, r \geq 0$. Among programs that complete all jobs with $Q \leq n + r$ on every execution, the minimum worst protected work is*

$$L_3^*(B, r) = \Omega_{\min((B-r)_+, n)}, \quad L_2^*(B, r) = \begin{cases} 0, & B \leq r, \\ \Omega, & B > r, \end{cases}$$

with and without cached completion, respectively. Any DAG is permitted.

PROOF. At most r calls may fail to complete. Fresh attempts have disjoint intervals between read and comparison; each failure needs a distinct write. If $B \leq r$ they give $Q \leq n + B$, $L = 0$. Otherwise switch after r failures to cached completion: at most $B - r$ writes remain, giving the stated bound. All-protected gives the two-mode upper bound.

For the two-mode lower bound when $B > r$, reject each cheap call with a fresh own-key write, stopping writes after the $(r + 1)$ st rejection. This uses at most $r + 1 \leq B$ writes. An admissible program cannot incur that many noncompleting calls, so all its jobs complete under protection.

For three modes with $B > r$, target the $k = \min(B - r, n)$ largest jobs. Reject their cheap calls and force their cached calls to recompute by fresh own-key writes immediately before comparison; fresh callbacks already incur w_i inside. Stop writes after $r + 1$ target cheap rejections. Before a next write, at most r such rejections and $k - 1$ cached target completions have occurred, so that write still uses at most $r + k \leq B$ writes. Admissibility excludes $r + 1$ noncompleting calls. Thus every target completes with protected work, giving $L \geq \Omega_k$. Fresh identities invalidate retained preparations, and no inspection or completed callback retains a guard across the chosen boundary. $\square$

THEOREM 4 (ONE POLICY FOR EVERY WRITER BUDGET). *Let $\mathcal{U}_j(r)$ contain the j -mode programs that receive no B and complete with $Q \leq n + r$ in every finite-write environment. Set $U_j(B, r) = \inf_{P \in \mathcal{U}_j(r)} \sup_{\# \text{writes} \leq B} L(P, E)$. Then*

$$U_3(B, r) = L_3^*(B, r), \quad U_2(B, r) = \Omega \mathbf{1}[B \geq r].$$

One policy per mode family attains these values simultaneously for all B and positive works on any DAG, using only readiness and a global cheap-failure counter.

PROOF. Switch after r cheap failures to a fresh read/preparation with cached completion (three) or to fresh protected completion (two). This uses at most r extra calls universally. At $B < r$ no protection is needed; at $B = r$ cached completions after switching still match. Theorem 3 supplies the remaining upper and three-mode lower bounds.

For two modes when $B \geq r$, reject every cheap call with a fresh own-key write until r rejections, then stop. Only protected calls complete before this. Afterwards, another rejectable call could fail in a different finite environment, violating the call cap. Thus all jobs complete under protection with at most $r \leq B$ writes. That additional hypothetical write is absent from the lower-bound execution. Retained preparations and inspections cannot prevent a fresh write before comparison. $\square$

For equal works and $B > r$, the ratio is $n/\min(B - r, n)$, reaching n at $r = B - 1$. Linear prices $W + \lambda L + \kappa Q$ correspond to $p_i = \lambda w_i$, $v_i = k_i = \kappa$; these counts do not estimate throughput.

Work cost of simultaneous protection optimality. Among $\mathcal{U}_3(0)$ programs attaining the optimal worst- L curve for every budget, minimum worst W at B is $\Omega + \Omega_{\min(B,n)}$. All-cached attains it with $Q = n$, $W = \Omega + L$. To prove necessity, target the $k = \min(B, n)$ largest jobs. Each counted call must complete. Immediately before each target's first comparison/acquisition, insert a fresh write exactly if its kernel was previously executed. Every target then executes inside. If this uses $h \leq k$ writes, the simultaneous guarantee gives $\Omega_k \leq L \leq \Omega_h$. Positivity forces $h = k$: each target pays a prior and a protected kernel. Unknown B alone does not force this cost; the artifact gives a counterexample.

For general r , consider the threshold family that freshly prepares before each attempt, uses cheap validation until r global failures, and then cached completion. It may adaptively choose another ready job after any failure, but retains no other preparation.

PROPOSITION 5 (WORK-OPTIMAL CHOICE WITHIN THE THRESHOLD FAMILY). *Minimum-work ready selection minimizes worst W within this family simultaneously for every B, r , even against selectors that receive B . A heap constructs its fixed order in $O(|E| + n \log(n+1))$ operations. This is not optimality among all causal programs or the full (W, L, Q) Pareto frontier.*

PROOF. Let $X(S, b, t)$ be optimal excess work over mandatory work, with t cheap failures left. At $t = 0$ it is the sum of the largest $\min(b, |S|)$ works in S ; at $b = 0$ it is zero. For $b, t > 0$,

$$X(S, b, t) = \min_{i \in \mathcal{A}(S)} \max\{X(S - i, b, t), w_i + X(S, b - 1, t - 1)\}.$$

If $T \subseteq S$, simulating omitted jobs as successful virtual attempts shows $X(T, b, t) \leq X(S, b, t)$: they consume neither failures nor slack. For ready i, j with $w_i \leq w_j$, compare choosing j first with choosing i , then j after success. Failure of i , success of i then failure of j , and both successes are each bounded by the j -first value, by weight order and subset monotonicity. Hence i is optimal at every state. Failure changes neither readiness nor weights, so the rule repeats the current job independently of b, t . $\square$

Certifying post-threshold protection choices. After r cheap failures, let h count cached mismatches and ℓ work already protected. Consider a boundary with unfinished jobs S , no retained preparation, and an observed prefix admitting exactly $r + h$ actual writes. This holds for the modeled serial histories; it does not compress histories revealing additional writes.

THEOREM 6 (EXACT SUFFIX FEASIBILITY). *A continuation with at most $|S|$ further calls meets Theorem 4's protection bound in every finite-write extension iff*

$$C(S, h, \ell) : \quad \ell + \Omega_t(S) \leq \Omega_{h+t}(J) \quad (0 \leq t \leq |S|). \quad (4)$$

PROOF. No further rejectable call is allowed: another adversarial write could force a failure beyond the cap. For necessity, target the t largest remaining jobs. A fresh callback already protects its target; otherwise one fresh own-key write before the eventual comparison invalidates its retained preparations, forcing protected recomputation. At most $u \leq t$ writes force $L \geq \ell + \Omega_t(S)$, while the guarantee requires $L \leq \Omega_{h+u}(J) \leq \Omega_{h+t}(J)$. Inspections cannot retain protection and noncompletion is inadmissible. For sufficiency, freshly prepare/cached-complete in any legal order. Its u mismatches concern distinct jobs, giving $L \leq \ell + \Omega_u(S)$. Every call completes. $\square$

Fresh completion of ready i is therefore feasible exactly when $C(S - i, h, \ell + w_i)$ holds. Cached matching preserves C by subset monotonicity; on mismatch use $w_i + \Omega_t(S - i) \leq \Omega_{t+1}(S)$. These finite inequalities certify allowable actions without B .

The *all-tail* rule finishes all remaining jobs freshly when $\ell + \Omega(S) \leq \Omega_h(J)$; otherwise it continues cached completion. A different rule greedily takes each certified fresh next job.

PROPOSITION 7. *Either rule retains the exact worst-L frontier and universal call cap, and weakly reduces worst W relative to the same-order threshold policy.*

PROOF. At the threshold switch $C(S, 0, 0)$ holds. Certified choices preserve it, every later call completes, and Theorem 4 gives equality. For work, replace each fresh completion in a run by a threshold cached match, retaining every other outcome. This same-order execution has equal W and no additional writes. $\square$

On chain $w = (2, 4, 1, 1)$, $r = 1$, $B = 4$, all-tail reduces worst W from 18 to 17 at $L = 7$, $Q = 5$; both return to 18 at $B \geq 5$. The two certified rules are incomparable (Section 8); feasibility does not establish general W -optimality.

5 All-Budget Compilation

Direct evaluation requires up to $(B + 1)N$ states, $N \leq 2^n$. Our binary-input construction removes budget enumeration; arbitrary DAGs still retain the ideal dimension.

5.1 Reducing the Self-Referential Choice

LEMMA 8 (SUBSET MONOTONICITY AND CHEAPEST VALIDATION). *If $T \subseteq S$, then $F(T, b) \leq F(S, b)$. For any minimum- c_i job $i \in \mathcal{A}(S)$, its cheap-validation option is no larger than that of any other available job.*

PROOF. For monotonicity, simulate a policy for S , completing omitted jobs without invalidation and with nonnegative excess. Relay retained jobs' actions and outcomes. Every virtually available retained job is physically available; omitted jobs spend no token. Physical cost never exceeds virtual cost. Equivalently, induction on $|S|$ and b compares corresponding terms of Equation (2).

At $b = 0$ all cheap-validation values are zero. For $b > 0$, let $j \neq i$ be available. It remains available after removing i . Choosing cheap validation of j in $S - i$ and using monotonicity gives

$$F(S - i, b) \leq \max\{F(S - \{i, j\}, b), c_j + F(S - i, b - 1)\} \leq \max\{F(S - j, b), c_j + F(S, b - 1)\}.$$

The other branch $c_i + F(S, b - 1)$ is also bounded by the last maximum. Their maximum is the cheap-validation value for i . $\square$

For a child curve f with $f(0) = 0$, define the insertion operator

$$(I_s f)(0) = 0, \quad (I_s f)(b) = \max\{f(b), s + f(b - 1)\}, \quad b \geq 1.$$

Fix a cheapest available i , put $g(b) = F(S - i, b)$, $c = c_i$, and

$$h(b) = \min_{j \in \mathcal{A}(S)} \{P_j + F(S - j, b), \delta_j + I_{s_j} F(S - j, \cdot)(b)\}. \quad (5)$$

At zero, $h(0) = \min_j \{P_j, \delta_j\}$ is allowed to be positive. Monotonicity and feasible completing modes give $g(b) \leq F(S, b) \leq h(b)$. Equation (2) becomes

$$f_0 = 0, \quad f_b = \min\{h(b), \max[g(b), c + f_{b-1}]\}, \quad b > 0. \quad (6)$$

This is a specialization of a reflected recurrence [28], not a new general dynamic-programming principle. Its particular child curves permit the following closed form.

LEMMA 9 (CONCAVE FLOOR AND CAP). *Suppose g, h are nonnegative, nondecreasing and discrete concave, $g(0) = 0$, and $h \geq g$. Define*

$$D(0) = 0, \quad D(b) = \sum_{t=1}^b \max\{c, \Delta g(t)\}, \quad c > 0.$$

Then Equation (6) has solution $f_b = \min\{h(b), D(b)\}$, which is nondecreasing and discrete concave.

PROOF. Since $\Delta g(t) - c$ is nonincreasing,

$$D(b) = \max_{0 \leq t \leq b} \{g(t) + c(b - t)\} = \max\{g(b), c + D(b - 1)\}.$$

Induct on b . If $h(b - 1) \geq D(b - 1)$, substitute this identity directly. Otherwise choose a maximizer t for $D(b - 1)$. Because $h(b - 1) \geq g(b - 1)$, the strict inequality requires $t < b - 1$. From $h(t) \geq g(t)$,

$$h(b - 1) - h(t) < c(b - 1 - t).$$

Concavity makes the next difference of h smaller than c . Thus the recurrence selects $h(b) < c + h(b - 1)$. The same block inequality for g makes $D(b) = D(b - 1) + c > h(b)$. This proves the formula in both cases, including the origin. Flooring preserves nonincreasing nonnegative differences. Pointwise minima preserve nondecreasingness and discrete concavity: minimize $2u_b \geq u_{b-1} + u_{b+1} \geq \min_u u_{b-1} + \min_u u_{b+1}$ over the candidate curves. $\square$

5.2 Integer Insertion and Supporting Slopes

We prove the cached maximum's concavity through its marginal structure.

LEMMA 10 (INTEGER MARGINAL INSERTION). *Let a bounded curve with $f(0) = 0$ have a nonnegative supporting representation $f(b) = \min_{d \in D} (db + q_d)$ for all integer $b \geq 0$, with finite $D \subseteq \mathbb{R}_{\geq 0}$ and $0 \in D$. For $s > 0$, define*

$$Q_s = \max_{b \in \mathbb{N}} \{f(b) - sb\}.$$

Then

$$I_s f(b) = \min \left\{ sb + Q_s, \min_{d \in D} (db + q_d + (s - d)_+) \right\}. \quad (7)$$

Its marginal sequence is that of f with one additional marginal s inserted in decreasing order.

PROOF. The marginals of f are nonincreasing and eventually zero. Let t be the number at least s . For $b \leq t$, $I_s f(b) = f(b)$; for $b > t$, it equals $s + f(b - 1)$. This proves insertion and gives $Q_s = f(t) - st$.

Every line on the right of Equation (7) majorizes this inserted curve. Before t , unchanged slopes at least s already majorize f ; smaller slopes receive a nonnegative shift. After t , shifting a smaller slope by $s - d$ exactly forms s plus its old value at $b - 1$, while a larger slope provides at least the same increase. The contact line $sb + Q_s$ majorizes both portions.

For attainment, an active supporting slope d_b at integer b satisfies

$$\Delta f(b + 1) \leq d_b \leq \Delta f(b),$$

where only the first inequality is needed at $b = 0$. At $b < t$ an active line therefore has slope at least s and is unchanged. At $b = t, t + 1$ the contact line attains. At $b \geq t + 2$, an active line at $b - 1$ has slope smaller than s and its shifted version attains. The cases $t = 0$ and $f \equiv 0$ follow from the same contact line. $\square$

The contact is over integers. For $f(b) = \min\{100, b + 1, 3b\}$ and $s = 2$, $I_s f(1) = 2$. Replacing the integer contact 0 by the continuous supporting envelope's supremum $1/2$ gives 2.5 in Equation (7). Computing a real contact and rounding afterwards is not the stated identity.

The floor operation also preserves a finite input-slope basis. If $f(b) = \min_d (db + q_d)$ and $c > 0$, then

$$\sum_{t=1}^b \max\{c, \Delta f(t)\} = \min \left\{ cb + Q_c, \min_{d > c} (db + q_d) \right\}. \quad (8)$$

Let $a = \#\{t \geq 1 : \Delta f(t) > c\}$, the first integer maximizing $f(b) - cb$. The floor equals f through a , and equals $cb + Q_c$ thereafter. At $b < a$ every attaining line has slope above c ; at and after a , each

retained larger-slope line stays above the contact line. This definition also handles a plateau with marginal exactly c , or $a = 0$, without special rounding.

THEOREM 11 (ALL-BUDGET BASIS AND ENCODED SIZE). *For every residual set S , $F(S, \cdot)$ is nonnegative, nondecreasing and discrete concave, with eventual value $\sum_{i \in S} P_i$. It has a supporting representation*

$$F(S, b) = \min_{d \in \mathcal{D}_S} (db + q_{S,d}), \quad \mathcal{D}_S \subseteq \{0\} \cup \{c_i, s_i : i \in S\}. \quad (9)$$

There are at most $2|S| + 1$ lines and at most $4|S|$ positive marginal runs. All reachable-state curves and action data can be constructed and checked in $2^n \text{poly}(L)$ bit time, with no iteration through the magnitude of B .

PROOF. Induct on $|S|$, starting with the zero line for the empty set. By Lemma 10, each cached term in h shifts existing intercepts and adds only slope s_j . Protected terms shift intercepts by P_j . Their minimum is a concave supporting envelope. Lemma 8 gives $g \leq F \leq h$ independently of the desired formula, so Lemma 9 applies. Equation (8) adds only the cheapest available c_i . Union the line families and retain the smallest intercept per slope. This proves Equation (9), concavity and nondecreasingness.

Protecting all jobs gives the bound $\sum_i P_i$. A bounded finite lower envelope of nonnegative slopes is eventually constant; let its limit be L_S . By smaller-set induction, sufficiently large budgets in Equation (2) satisfy

$$L_S = \min_{i \in \mathcal{A}(S)} \left\{ P_i + \sum_{j \in S-i} P_j, \quad c_i + L_S, \quad \delta_i + s_i + \sum_{j \in S-i} P_j \right\}.$$

Here $c_i > 0$ and $\delta_i + s_i = P_i + w_i > P_i$, forcing $L_S = \sum_{i \in S} P_i$.

At most $2|S|$ distinct positive slopes give at most that many continuous-envelope changes. Integer sampling adds at most one bridging unit interval per noninteger change, so at most $4|S|$ positive runs suffice. This is a conservative upper bound, not a tightness claim. Summing $4|S| + 1$ over all subsets gives $(2n + 1)2^n$ profile entries, with at most $n2^n$ action identities.

Generated intercepts obey $0 \leq q_{S,d} \leq \sum_{i \in S} (P_i + w_i)$. A protected shift adds P_i ; a cached shift adds at most $\delta_i + s_i = P_i + w_i$; a new contact is bounded by child saturation; flooring retains old intercepts. Contacts require no unbounded search: for profile triples (a, y, d) ,

$$Q_s = \max_{(a, y, d) \in \text{profile}(f)} (y - sa).$$

Each piece is affine and the final constant tail decreases after subtracting sb , so a listed breakpoint attains the maximum. Intercepts, integer slopes and crossing-neighbor positions consequently have polynomial bit length in L . Each state uses polynomially many shifts, contacts and envelope intersections on $O(n)$ lines. Affine-partition checking below has polynomially many such endpoints per state. Enumerating at most 2^n states proves the bit bound. $\square$

5.3 Emitted Policies and Independent Checking

Profiles store affine pieces (a, y, d) with a constant tail. Both constructors process reachable upsets child-first, using floor/cap or supporting-basis operations without enumerating budgets. Runtime selection evaluates Equation (2) with $O(n)$ indexed profiles, chooses a deterministic argmin and decrements allowance on mismatch. Query cost is polynomial in n , stored size and $\log(B + 1)$; optimal ties need not imply pointwise dominance.

The checker imports no constructor. It reconstructs readiness and the reachable domain, validates raw and derived prices, baseline, available-job lists, cheapest-fast IDs, origins, continuity, decreasing slopes and tails, and verifies Bellman equality. Partition at own/child starts and their one-step

shifts. Within each interval all atomic expressions are affine. Checking endpoints and floor/ceiling neighbors of all crossings proves equality throughout; check the constant final interval directly. Compare the supporting basis with its emitted curve by the same affine method.

Induction on $|S| + b$ certifies values and the mathematical argmin. Executable selection, arithmetic, parsing and the operational abstraction remain trusted or separately tested. Earlier routing-binding failures remain recorded in the artifact and repaired by RQ1's field checks. Polynomial-per-state bounds concern generated certificates; arbitrary encodings additionally cost their supplied length. Checking does not establish application-price adequacy.

What a requested-value query can skip. Large b need not require a profile. Classical concave allocation [13] gives the following charged-model bounds:

$$\max_{\substack{b_i \in \mathbb{N} \\ \sum_i b_i \leq b}} \sum_{i \in S} \min(P_i, b_i c_i) \leq F(S, b) \leq \min_{t \in \{0\} \cup \{c_i : i \in S\}} \left(\sum_{c_i > t} P_i + bt \right). \quad (10)$$

For the lower bound, assign private failure budgets b_i . Protected or cached-failure completion pays at least P_i ; any other completion after exhausting its private allowance has already paid $b_i c_i$ in cheap failures. Their total allowance is at most b . For the upper bound, protect jobs with $c_i > t$ and retry the others in any topological order. Their failure costs are at most t . The allocation merges compressed marginal runs: $\lfloor P_i / c_i \rfloor$ copies of c_i , then $P_i \bmod c_i$ if nonzero. It never iterates to b . Equality certifies a value directly, and $b \geq \sum_i \lceil P_i / c_i \rceil$ guarantees the constant tail $\sum_i P_i$. Our requested-value comparator uses these tests at every subproblem and separately certifies equality leaves and concave peaks. It returns requested values, whereas the full artifact additionally supplies all-budget profiles and routing data.

6 Compact Compilation, a Sharp Ratio and the General Barrier

The general compiler handles arbitrary dependencies and prices. A price condition, however, permits a stronger result even with positive call charges and an essential cached mode.

6.1 Reducing Mixed Charges to Two Fees

For a concave suffix f with $f(0) = 0$, write

$$R_{a,q}f(b) = f(b) + \min \left\{ q, \sum_{\ell=1}^b (a - \Delta f(\ell))_+ \right\}. \quad (11)$$

This is the two-fee operator with failure price $a > 0$ and protected premium $q \geq 0$: by Lemma 9, it equals $\min(f + q, L_a f)$, where $L_a f(b) = \max_{0 \leq j \leq b} \{f(j) + a(b - j)\}$. It is monotone in f .

LEMMA 12 (TWO-FEE ORDERING AND PACKING). *Independent two-fee jobs admit a fixed optimal nondecreasing- a completion order, including against adaptive choices. Composing their R operators in this order gives their value H . Reverse composition fills marginal deficits below the next a : all positive suffix marginals except possibly the last are at least a . Each job adds at most two runs. Every sorted suffix is the root curve capped at its remaining premium sum.*

PROOF. Let G be the optimum restricted to the sorted order. Inserting a protected job i at its sorted position gives $G(S, b) \leq q_i + G(S - i, b)$ without spending a token. Induct on $|S| + b$. The first sorted job has minimum failure price; the proof of Lemma 8, with only protected and cheap modes, makes its cheap option no larger than any other's. The smaller-set and previous-budget values equal G by induction. Thus G is bounded above by every unrestricted Bellman option, while feasibility gives the reverse inequality.

Equation (11) preserves marginals at least a and raises smaller ones toward a using at most q mass. In reverse sorted order, only the final positive singleton can be below a . Raise it first; write any remaining mass as $ha + r$, $0 \leq r < a$, and append h copies of a and one r if nonzero. Constantly many run operations suffice. Removing the added tail mass recovers the previous suffix, proving the cap identity. $\square$

THEOREM 13 (EXACT MIXED-PRICE REDUCTION). *Suppose $p_i \delta_i = 0$ for every job, with the original prices from Equation (1), and set*

$$t_i = \begin{cases} \min(c_i, s_i), & \delta_i = 0, \\ c_i, & \delta_i > 0 \quad (\text{hence } p_i = 0). \end{cases} \quad (12)$$

For independent jobs, the charged excess $F(S, b)$ equals the two-fee optimum $H(S, b)$ with prices (t_i, P_i) at every integer budget. A fixed nondecreasing- t completion order is optimal. The same equality holds for a DAG satisfying $t_u \leq t_v$ on every edge $u \rightarrow v$, with ties ordered topologically. The total charge remains $\sum_{i \in S} c_i + H(S, b)$.

PROOF. Marginal insertion (Lemma 10) commutes with deficit filling when $s \geq a$:

$$R_{a,q} I_s f = I_s R_{a,q} f. \quad (13)$$

Indeed, insertion of s precedes every marginal below a and adds no deficit; $R_{a,q}$ changes only those later marginals, raising them at most to a .

If f is a sorted suffix with effective costs at least t , $s \geq t$ and $q < s$, then $R_{t,q} f \leq I_s f$. Before the final partial marginal the deficit is zero. At that marginal the increment is at most $t - \Delta f(b) \leq s - \Delta f(b)$; after saturation it is at most $q < s$. These cases also show $R_{s,q} f = \min(f + q, I_s f)$ when $t = s$.

For $\delta_i > 0$, $p_i = 0$, so cached completion is dominated: $\delta_i + I_{s_i} f \geq P_i + f$. The ordered charged operator is exactly R_{c_i, P_i} . For $\delta_i = 0$, $P_i = p_i < s_i$, and it is $\min(f + P_i, L_{c_i} f, I_{s_i} f)$. The preceding suffix inequalities make this R_{t_i, P_i} , both when $t_i = c_i$ and when $t_i = s_i$. Thus the sorted charged policy attains H .

For a matching lower bound, the two-fee Bellman equation bounds $H(S, b)$ by $P_i + H(S - i, b)$ and by $\max\{H(S - i, b), c_i + H(S, b - 1)\}$, since $t_i \leq c_i$. These cover protected and cheap choices. Positive- δ_i cached choices are covered by protection dominance. For a zero- δ_i job at any sorted position, write $H(S) = AR_{t_i, P_i} f$ and $H(S - i) = Af$. Every operator in the preceding composition A has cost at most $t_i \leq s_i$. Monotonicity, the suffix inequality and Equation (13) give

$$H(S) \leq AI_{s_i} f = I_{s_i} Af = I_{s_i} H(S - i).$$

Hence every original charged Bellman option is at least H ; induction on $|S| + b$ proves the lower bound against adaptive choices. Finally, removing DAG edges gives an independent lower bound, and a compatible sorted topological policy attains it legally. $\square$

The class mixes $k_i \leq v_i$ jobs with zero-premium jobs. With $v_i = k_i = \kappa$ and $p_i = \lambda w_i$, $t_i = \min(w_i + \kappa, (1 + \lambda)w_i)$ increases with w_i , so work order suffices. Effective prices preserve cached choices; the total-cost baseline remains $\sum c_i$.

6.2 A Checked Cursor Policy

Minimum- t Kahn traversal finds a sorted topological order when the edge condition holds. Apply Lemma 12's packing to (t_i, P_i) , retaining the original charged input. For sorted suffix S_k , root curve H and $Z_k = \sum_{i \in S_k} P_i$, store

$$F(S_k, b) = \min\{H(b), Z_k\}. \quad (14)$$

The artifact stores $O(n)$ entries beyond the graph for its $n + 1$ policy suffixes; the general artifact covers all reachable unfinished sets.

The checker independently recomputes reduction, baseline and order; validates origin, positive integer run lengths, decreasing heights, zero tail and total mass $H(\infty) = Z_0$; and checks the two-fee Bellman equations. For adjacent caps $z = Z_k$, $a = Z_{k+1}$, let u and v be the first budgets where H reaches a and z . Check the uncapped prefix, the boundary u , the constant marginal t_i between u and v , and saturation at v ; concavity extends marginal endpoint checks across each interval. Zero premiums require identical adjacent caps. The terminal cap is zero. These conditions verify all budgets by suffix induction; Theorem 13 supplies the charged/global-optimality reduction. Additional original-mode boundary checks are consistency controls, not a substitute for that proof.

Construction takes $O(|E| + n \log(n + 1))$ arithmetic operations. Checking a supplied root with r runs takes $O(|E| + n \log(r + 2) + r)$, plus its encoding length. Generated roots have $r \leq 2n$, giving the same asymptotic bound. Binary operand lengths still count. Prefix lengths/areas give $O(\log(n + 1))$ value queries. At a cursor, evaluate the three original branches in Equation (2) for its fixed next job using adjacent caps. Cheap failure retains the cursor; cached failure spends one token and advances it. All paid charges remain original. Inputs failing the price or edge condition use the general charged compiler.

The conditions are sufficient. For independent $w = (1, 1, 1, 2)$, $p_i = w_i$, common $v = 5$, $k = 6$, and $B = 1$, ascending work order costs 31, while order $(2, 1, 1, 1)$ and the adaptive optimum cost 30; here $p_i \delta_i > 0$. Even with $\delta_i = 0$, sorting by c_i can fail: $(w, p, k, v) = (5, 6, 5, 8)$, $(5, 4, 6, 8)$ gives $c = (10, 11)$ but $t = (10, 9)$, and $B = 1$ totals 31 versus 30. Both refutations remain in the artifact.

6.3 A Sharp Bound on Independent-Job Savings

The third mode's scalar benefit is bounded even outside the compact class. Write F_2, F_3 for the two- and three-mode excess values and $A = \sum_i c_i$.

THEOREM 14 (SHARP INDEPENDENT-JOB RATIO). *For independent jobs with arbitrary charged prices and any integer $B \geq 0$,*

$$\frac{A + F_2(B)}{A + F_3(B)} \leq \frac{1 + \sqrt{2}}{2}.$$

The constant is a supremum, already approached with common $v_i = k_i$ and $p_i = \lambda w_i$.

The supplementary packing proof establishes the bound and tightness. The supremum saving is $3 - 2\sqrt{2} \approx 17.16\%$ for independent inputs; no corresponding DAG bound is asserted.

6.4 Hardness Outside the Compatible Class

Set $v_i = k_i = 0$. Then $c_i = w_i$, $\delta_i = 0$, $P_i = p_i$, and the cached mode is dominated by a cheap attempt followed by immediate protection on failure. Equation (2) reduces to

$$V(S, b) = \min_i \{p_i + V(S - i, b), \max[V(S - i, b), c_i + V(S, b - 1)]\}.$$

At $B = 1$, a topological order π and the protected jobs H on its no-failure path have worst excess

$$C(\pi, H) = \max \left\{ \sum_{j \in H} p_j, \max_{i \notin H} \left(c_i + \sum_{j \in H: j \prec_{\pi} i} p_j \right) \right\}. \quad (15)$$

After the one possible failure, every remaining job finishes without excess. Minimizing this expression is exact, so an initially fixed order suffices at one failure.

THEOREM 15 (SINGLE-FAILURE HARDNESS). *The zero-fee subcase of deciding $F(J, 1) \leq K$ is NP-complete with at most two DAG layers, maximum indegree two, unit premiums and at most two distinct positive failure prices.*

PROOF. Equation (15) verifies order/mode bits in polynomial time. For nontrivial CLIQUE [34], $k \geq 2$, $m \geq \ell = \binom{k}{2}$, add an isolated vertex so $n > k$. Set $K = n + m - \ell > k$; give vertices $(c, p) = (K + 1, 1)$ and endpoint-dependent edges $(K - k, 1)$. A clique gives value K : protect its vertices, run its ℓ edges fast, protect the remainder. Conversely, value $\leq K$ forces all vertices protected and every fast edge among the first k protected vertices. No-failure cost $n + m - f \leq K$ requires at least ℓ fast edges, hence a clique. All numbers are $\leq n + m + 1$. $\square$

This does not establish NP membership for binary B or require ideal enumeration. A w -chain cover bounds ideals by $\prod_{\ell} (n_{\ell} + 1)$, polynomial for fixed w (XP).

Dependencies can require adaptive ordering even without fees: $c = (3, 7, 5, 1)$, $p = (2, 2, 6, 2)$ and edges $0 \rightarrow 1, 0 \rightarrow 2, 1 \rightarrow 3$ give 24 versus 26 at $B = 2$. Every fixed order gets optimal adaptive modes and knows B . This constructed separation does not establish typical benefit.

7 Source Integrations and Their Boundaries

Roslyn document updates without a writer budget. The update loop in `Workspace.cs` at commit 6c4a46a3 captures the current immutable solution, transforms it outside a semaphore, then validates reference identity under that semaphore. A mismatch retries; publication preserves notification ordering [2]. We modify only the `SourceText` document-update path. A foreground-owned state spends r failures globally across four requested replacements. After exhaustion, the two-mode variant transforms the latest solution under protection; the three-mode variant prepares outside, reuses on a match, and recomputes the latest input under protection on mismatch. Other updates retain the original path. The selector receives neither writer count nor source-size/price tables.

Normal transformations clear their scratch document list each time. Linked text propagation, immutable solution publication and synchronous/asynchronous notifications retain the source ordering. Captured solutions remain valid after subsequent independent-document writes. This is a transformation-count correspondence under normal, nonreentrant execution, not the pure fixed- w contract: lazy state, allocation and `VersionStamp` generation prevent transferring scalar minimax values. A same-text notification reentry test exposed a deadlock in the initial two-mode patch. A source-derived cached-text identity guard restores the pre-lock no-op; the old timeout is retained. Changed-text reentrant hosts remain outside the evaluated contract.

Deephaven with owned refreshes. For `ConstructSnapshot/hierarchical-table` sources at 6367313a, the server API materializes key directives K followed by viewport V [5]. Fixed keys/hierarchy, stated Action changes and a 16-row/four-column viewport yield source-derived selected-operation bounds $W_K = 4$, $W_V = 104$, including mid-body updates; the full derivation is supplementary. Completed K persists for V after live Actions change. With protected weight $1 + \lambda$, supply $p_i = \lambda W_i$ and zero call charges. This excludes allocation, waiting and unselected API instructions.

The caller owns at most q public refresh requests [7]; synchronized START/completion boundaries charge at most $2q$ failed bodies. Periodic updates do not supply such a bound [6], and partially notified interleavings are excluded. Imported compilation gives an upper bound: at $\lambda = 3$, $q = 2$ a script attains 424 versus a same-budget local rule's 432, but a phase-aware game gives 420. The abstract game is therefore not the native system's exact minimax game.

8 Implementation and Evaluation

We test game/policy agreement and collateral blocking (RQ1), source integrations (RQ2), and construction costs (RQ3). Studies are authored and exploratory: reused cases are identified and new generation rules precede outcomes. Every input, failure, timeout, invalid unit and correction remains recorded; none is a held-out application population.

Experiments use an Apple M2/24 GiB, Python 3.10.9 (later 3.12.14) and Homebrew OpenJDK 17.0.19. Retained ConcurrentHashMap.java matches the cited source; the binary was not rebuilt. Hashes bind inputs, code, profiles, commands and traces. Separate Python checkers validate outputs/equations. Codex assisted model/experiment design, input and checker construction, source comparison, proof exploration, data analysis and writing; a Claude consultation informed study direction. Reported traces come from archived executions. These author-side checks are neither mechanical proof nor independent blind review; no human or simulated-human study was performed.

8.1 RQ1: Charged Values, Routing and Native Calls

The general compiler agrees with direct values on all 3,183 observed inputs after serialization and reloading, including 128-bit prices. All 19,485 compact/fallback inputs agree (15,727 observed, 3,758 new): 16,770 compact routes, 2,715 fallbacks and 527,409 checked policy branches. Certificate/runtime/CLI controls preserve repaired numeric-alias and null-input binding defects.

For 34 native inputs, all 2,278 paths, 816 concurrent runs and 340 maxima agree. Earlier epoch, output and writer-order defects are retained; source witness replay checks all 3,098 records and 23 corruptions. The compact adapter qualifies for six of 36 screened inputs plus 20 new cases: all 610 native records and 208 maxima agree, including positive- δ cap 8 versus two-mode 9. The 30 inapplicable rows remain. Native arithmetic is bounded.

Separate resource guarantees. Eight shapes, two call weights and six selectors give 2,442 native records and 576 group maxima, all matching W, L, Q and $R = W + L + 8\kappa Q$. Scalar three-mode gains in only 2/96 cells (72 versus 80); 94 tie. A decreasing five-job chain has scalar-optimal $L = 48$ versus cached-all's 40 at equal $Q = 5$ and worst cost 200. The complete inputs and source-counter reconstruction are retained.

A repaired missing-precondition defect passes a strict recheck of all 96 tables/2,442 saved decisions; five controls also reject unrelated errors masquerading as the intended native rejection.

Direct L equations agree on 265,629 budget/slack pairs from 5,421 DAGs. Budget-aware/unaware selectors give 9,314/17,714 native records and 1,024/1,280 frontier maxima. The unaware bytecode receives no writer/price/table state; native budgets reach 4. Strict record/parser controls and later work reconstruction pass. All 86,662 orders/2,773,184 pairs on 5,519 inputs confirm fixed-order greedy optimality; adaptive-family equations agree on 176,608 roots, $B = 0..7, r = 0..3$ (Proposition 5). These later equations add no native measurements.

All-tail checks cover 5,520 inputs/176,640 roots: 285 strict worst- W improvements and 176,355 ties, all preserving the frontier. Its 2,552 Java paths match 144 maxima. The suffix extension adds 1,024 five-job chains over works $\{1, 2, 3, 4\}$: across 6,544 inputs/209,408 roots, greedy certified choice is better/worse/tied versus all-tail at 1/3/209,404. A separate finite-rank AND-OR recursion agrees with Equation (4) on 665,500 states for 363 weight tuples, with arbitrary or fixed job choice. All 20,594 suffix Java paths match 384 maxima. Both native checkers reject eight corruptions; the suffix checker precedes its run but derives from the observed checker. Selectors receive only weights, predecessors, mode/ r and own outcomes. Selected iterations exclude parent mixing/allocation; all adverse results remain.

Ratio checks cover 13,295 inputs/172,835 roots and seven tightness families. Another 622 inputs at three scales give 24,378 roots and 3,732 large-budget queries, all successful. Scaling is not independent sampling. The 65,536-job family reaches 1.20545; the proof gives the limit.

A failed mean-time calibration. JMH 1.37 [3] tests nine wrappers, four payloads and two layouts: all 72 cells/144 forks/720 measurements complete. Each fork uses three 200-ms warmups and five 200-ms measurements. A fixed nonnegative w, v, k, p fit admits only 2/8 settings at maximum residual $\leq 20\%$; every unconstrained protection residual is negative and every fitted p is zero. Only 72 of 1,152 scalar roots belong to admitted states; none gains, while 1,080 remain outside. Different reuse, lifetimes and short warmups prevent establishing mean-time benefit.

Measured collateral blocking at the universal call cap. A Java experiment uses four dependent kernels with iteration weights (1, 2, 3, 5)s, $r = 0, 1, 2$ and exactly $B = r$ initial failures. Both budget-unaware policies have $Q = 4 + r$, $W = (11 + r)s$; protected iterations total 11s for two modes and zero for three. The metric excludes constant parent mixing/allocation. A two-mode policy given $B = r$ could also avoid protection.

At each completing callback, a probe timestamps/signals readiness before an unchanged-value compute on another key. The foreground holds its bin while awaiting readiness, then executes its body. Same-bin and disjoint-bin layouts use identical hooks; issue-to-entry time includes synchronization/scheduling. Four scales, three r values, two layouts/modes and 30 repetitions in each of three serial JVMs give 4,320 measurements plus 1,440 warmups. All complete and match 12 Python output references, with no exclusion/retry. A later cross-job/timing checker repair passes all raw records and rejects 12 corruptions.

Table 2 sums four probe waits. Same-bin medians at the largest scale drop from about 26.8 ms to 31.2 μ s; disjoint differences are submicrosecond and sometimes reverse. All 24 cells, paired differences, descriptive bootstrap intervals and fork medians are available. This constructed arrival opportunity supports conditional collateral blocking, not natural-arrival performance, WCET, whole-application speedup or blocking of get.

Table 2. Probe wait sums in μ s. Each range spans the three r -specific medians (90 invocations per policy/setting). Last column spans the three medians of paired two-minus-three differences for the disjoint control.

Scale s	Same-bin two	Same-bin three	Disjoint difference
64	30.5–31.0	29.8–30.1	0.001–0.063
2,048	61.6–62.9	29.0–29.2	0.021–0.209
65,536	884.1–885.3	29.0–29.4	0.041–0.293
2,097,152	26,813–26,825	31.2–31.3	–0.291––0.061

8.2 RQ2: Repository Text Updates and Imported Snapshot Bounds

Roslyn source execution. Pinned net8 Roslyn assemblies run under a pinned .NET 10 RC driver/runtime. An authored 1/8/64/256-project grid covers linked/no-op/equal-content/missing-document/deletion cases: all 204 revised-patch cases and 16 zero-writer unmodified controls pass. The initial patch’s 3,960 measurements and later reentry timeout remain separate; repaired reentry completes for all variants and unmodified binaries.

A second harness projects four upstream projects’ evaluated Compile items: 3,837 files, 4,019 documents, 40.8 MB; two package-external items are excluded before outcomes. The target maximizes sharing then size; the background is the largest single-project file. Membership, hashes

and selection are fixed beforehand. This is a source projection with internal references, not an editor/compiler benchmark. Four target and B background updates append comments. Checks cover initial texts, all 4,016 unaffected final text identities, captured linked contents, notification content and event count/order; event solution payloads are unchecked.

After 69 development cases and four zero-writer unmodified controls, three fresh processes each run two warmup and ten measured repetitions of all 33 mode/ r / B settings: $r = 0, 1, 2$, $B \in \{0, r, r + 1, r + 4\}$ with duplicates removed. Groups/modes are shuffled; an independent writer acts at the first B pre-lock opportunities. All 990 measured and 198 warmup units pass the trace checker and its 11 corruptions. Foreground semaphore admissions are $4 + B$ for original and $4 + \min(B, r)$ for both capped variants; the B writer admissions are excluded. Equal writer totals do not imply identical intermediate interleavings. Two-mode protected transformations are 4 if $B \geq r$, otherwise 0; three-mode counts are $\min((B - r)_+, 4)$. Across 330 measured batches/mode, calls are 2,040/1,590/1,590 and protected transformations 0/1,080/450 for original/two/three. Three-mode total transformations equal original (2,040), exceeding two-mode (1,590). For each comparator, three-mode foreground paired medians are lower in four settings and higher in seven. The counts transfer; elapsed-time improvement does not follow.

Deephaven boundaries. All 216 prefix/END cells succeed, with 153 cycles/223 inconsistent bodies but no prefix abort: ExpandAll bypasses the check. Compiler/local wins occur in only 3/36 cells; six lose to the upstream replica and four to the unknown-budget threshold. Twelve public-graph cells expose policy-dependent refresh counts. Changing Actions gives 28 full/eight contracted results; two retain old directives after re-expansion. The repaired checker rejects 17 corruptions (eight missed earlier); ten regressions and 108 phase-aware cells pass. Full charges, source conditions and corrections remain; native minimax is unestablished.

8.3 RQ3: Construction Cost and Stronger Value Queries

The general comparison uses 24 authored DAGs, $n = 8, 12$, three price scales and 1/4/12 queries through 2^{64} . ALL writes, reloads and checks all profiles; PEAK checks requested-value certificates; DP uses two Bellman rows. Cold imports-through-checking costs have 4-second/1-GiB caps.

Initial successes are 63/72 ALL, 62/72 PEAK and 64/72 DP; eight generic AND-OR units also succeed. Later stronger bounds give PEAK/DP 64/72 each. All other units time out; shared values agree. On 14 common 12-query successes, PEAK/ALL median time falls from 7.16 to 0.576; DP/ALL is 0.457. Seven long-premium ALL cases finish while both value methods time out. All nine ALL timeouts are independent 12-job cases.

Compilation on the compatible charged class. Campaign A has 42 compatible inputs, $n = 8..32768$, independent or height-two DAGs, three price scales and 1/12 queries. PACK writes, reloads, checks and indexes; ROOT omits those services. PEAK/DP validate adjacency and scan bounds. All 420 units record under 8-second/1-GiB caps (Table 3). PACK's 12 largest-input runs take 0.837–1.151 seconds, peak RSS 122,044,416 bytes. Median PEAK/PACK and DP/PACK ratios are 0.860/0.792: requested values can be cheaper.

Campaign A's large curves have only 40 runs. Campaign B fixes 18 inputs at $n = 512, 4096, 32768$, exactly $2n$ runs, long plateaus and boundary queries through 2^{64} . All 180 units record under the same caps. At 32,768 jobs, PACK produces/checks 65,536 runs in 1.333–1.603 seconds, peak RSS 152,911,872 bytes; median PEAK/PACK and DP/PACK ratios are 0.301/0.206. Shared values agree. An aggregation repair reused preserved raw records; no measurement was rerun. These structural single samples do not establish prevalence.

All 7,866 charged-order roots on 874 inputs succeed; only the known example gains from adaptation (24 versus 26), with none among 840 new independent inputs. Sixteen post hoc cases refute c

Table 3. Compact-class campaigns. Each cell is success/timeout/memory limit. A: 84 units per method; B: 36 units per method with $2n$ root runs. All outcomes are retained; returned services differ.

Method	A	B	Returned service
PACK	84/0/0	36/0/0	Checked all-budget cursor policy
ROOT	84/0/0	36/0/0	Same-reduction values, no checker
ALL	22/56/6	0/30/6	Checked all reachable unfinished sets
PEAK	34/50/0	18/6/12	Checked requested root values
DP	34/46/4	18/15/3	Requested values only

order. Separate zero-free gains occur at only 2/192 roots. CP-SAT/screened DP/full construction finish 78/68/53 roots; remaining roots time out. Complete denominators and outcomes remain available.

9 Related Work

Synchronization synthesis and fallback. Kung and Robinson [1], Section 3.3/p.220, already address repeated validation failure by retaining the critical-section semaphore during reexecution. Cached fallback is therefore an inherited mechanism; Theorem 3 gives its exact resource boundary for the stated call interface. Černý et al. [19] already choose optimistic retries, guards and caches. Their Theorem 6 preserves limit-average values through matched distributions/weights; an erratum supersedes the original hardness claim. Our potential establishes initial worst-total-cost equality for charged calls and actual writes. Their later work [20], Sections 6.1–6.2, already optimizes protected-statement counts and profiles contention/acquisition costs. Our call-cap frontier is worst- L optimal for every writer budget without receiving it; performance calibration remains open. Weighted reachability already expresses optimal worst fault cost [8]; resilience/cost synthesis and general game solvers are established [10, 11].

Reduction and robust dynamic programming. Gangamreddypalli et al. [21] preserve failures and reach original successful final states or failure. Our calls do not commute unconditionally with same-key writes; the potential gives no exact intermediate-state quotient. A cost-preserving reduction remains unestablished. Mannor et al. [9], Section 4/Theorem 5, provide finite global-deviation-budget DP: unfinished upsets, completing nominal branches and token-consuming failures instantiate it with negated cost and horizon $n + B$. Postek et al. [31] supply exact finite-candidate DP, state expansion and causal budgeted routing. Normal form and all-budget closure require additional arguments.

The zero-free package [35] supplies the reused hardness, floor/cap lemma and two-free ordering/packing. Its lineage map separates the atomic-call potential, universal resource frontier, second slope, sharp ratio and mixed-price reduction.

Envelopes, reflection and allocation. Burdzy et al. [28], Equations (2.12)–(2.13)/Theorem 2.6, give the general reflected recurrence; our floor/cap lemma specializes it to concavity. Marginal insertion is elementary. Bertsimas and Sim [32], Theorem 3/Equations (15)–(19), already give an input-deviation slope envelope $Z(\Gamma) = \min_{\ell} \{\Gamma d_{\ell} + F_{\ell}\}$. Such envelopes alone are not new. Here Equations (7)–(9) close the operations for repeated causal failures, two completing modes and dependent job choices. Reduction to a fixed binary robust set remains unestablished. Concave allocation [13, 14] supplies lower bounds; decision-dependent information and controllable uncertainty are broader settings [16, 17].

Scheduling and hardening. At one failure, the zero-free problem couples processing/tail modes $(p_i, 0)$ and $(0, c_i)$. Fixed modes use Lawler’s precedence sequencing [29]; our hardness concerns selecting modes. Izosimov et al. [22] activate DAG schedules under bounded failure histories with signalling/transparency constraints; An and Yang [12] synthesize stochastic hardening under fixed priorities. We omit those constraints. Chrobak et al. [15], Lemma 4/Theorem 5, concentrate faults on a critical job and compute prescribed-order latest completion in $O(n)$ independently of the fault bound. Their model mandates retry; ours chooses priced protection/cached completion and returns all-budget counted-work policies. Bold and Goerigk [33] use $\Gamma + 1$ DAG layers and a compact MIP after duration observation. Their activities complete despite delay; our failure leaves a job unfinished before another causal choice.

Ordering and rent-or-buy. Heterogeneous thresholds and ordering appear in ski rental [23, 27]. The direct map $f(H) = \sum_{i \in H} p_i$, $g(H) = \sum_{i \in H} c_i$, $T = B$ gives 6 for $(p, c) = (2, 5), (4, 6)$ at $B = 1$, versus our zero-free value 5. This excludes only that map: shared interference, mandatory work and causal completion must be matched. At $B = 1$, charged fixed-order excess is $\psi_{\pi_1} \circ \dots \circ \psi_{\pi_n}(0)$ with $\psi_i(x) = \min\{x + P_i, \max(x, c_i), \delta_i + \max(x, s_i)\}$. Kawase et al. [4] establish composition ordering and tractable/hard scheduling classes. This inherited scalar view does not directly resolve our nonconvex/nonconcave maps or all-budget closure.

10 Limitations and Threats to Validity

The theorem requires the whole-kernel, captured-parent, persistent-completion interface of Section 2. Partial/incremental work, replenished budgets, structural changes, exceptions and external milestone completion are excluded. A free certificate that all future writers have finished can reduce cost to baseline; the map supplies neither this certificate nor calibrated prices. Scalar minimax values use a supplied B ; Theorem 4 removes that input only for the stated resource objective.

The zero-free interface permits individual/batch acquisition and retained guards, with two-mode equality. Charged failed-acquisition experiments contain five strict improvements among 384 inputs, including unrestricted 242 versus macro 243. Immediately releasing or completing a failed guard is also refuted at an intermediate state (43 versus 45). Charged exposed-guard equality from all-unprepared states remains unproved despite finite agreement; the atomic-call theorem does not resolve it.

Roslyn results concern the instrumented nonreentrant text-update path; source-projected projects do not establish editor throughput or arbitrary host safety. Deephaven additionally requires owned finite refreshes and synchronized boundaries. Ordinary Web keys are static; equal clock parity can hide different notification states. Whole-system optimality and application prevalence remain unknown.

Authored tests do not mechanically verify proofs. Gains are sparse and direct methods sometimes avoid construction timeouts. Calibration finds neither positive premiums nor timing benefit. Probe arrivals are forced with hooks on one host; the effect is conditional. Human benefit and developer effort were not measured.

11 Conclusion

Charged atomic completion yields an exact normal form, a resource frontier and exact suffix feasibility without a writer-budget input, and all-budget compilation with a compact mixed-price class. A sharp ratio bounds independent-job savings. Roslyn source execution demonstrates capped calls and reduced protected transformations without general timing gains; controlled Java probes establish a conditional collateral-blocking effect.

Data Availability

The charged supplement [36] contains complete fixed inputs, proofs, sources, raw outcomes, corrections and replay commands; prior zero-fee evidence [35] is unchanged. Original/public hashes record path projections. Licensed modified sources retain notices; upstream archives, runtimes, caches, articles and private correspondence are excluded.

References

- [1] H. T. Kung and John T. Robinson. 1981. On optimistic methods for concurrency control. *ACM Transactions on Database Systems* 6, 2, 213–226. <https://doi.org/10.1145/319566.319567>.
- [2] .NET Foundation. 2025. *Roslyn source*, commit 6c4a46a31302167b425d5e0a31ea83c9a9aa1d09. *Workspace.cs*, *SolutionCompilationState.cs* and *SolutionState.cs*. Accessed September 9, 2026. <https://github.com/dotnet/roslyn/tree/6c4a46a31302167b425d5e0a31ea83c9a9aa1d09>.
- [3] OpenJDK. n.d. *JMH: Java Microbenchmark Harness*, version 1.37. Accessed September 9, 2026. <https://github.com/openjdk/jmh/tree/1.37>.
- [4] Yasushi Kawase, Kazuhisa Makino, and Kento Seimi. 2016. Optimal Composition Ordering Problems for Piecewise Linear Functions. In *ISAAC, LIPIcs* 64, 42:1–42:13. <https://doi.org/10.4230/LIPIcs.ISAAC.2016.42>. Extended version: *Algorithmica* (2018), <https://doi.org/10.1007/s00453-017-0397-y>.
- [5] Deephaven Data Labs. 2026. *Deephaven Core source*, commit 6367313a319b79437d4d2116c77836ad56d5138d. *Files ConstructSnapshot.java*, *HierarchicalTableImpl.java*, *EventDrivenUpdateGraph.java*, and *JsTreeTable.java*. Accessed September 9, 2026. <https://github.com/deephaven/deephaven-core/tree/6367313a319b79437d4d2116c77836ad56d5138d>.
- [6] Deephaven Data Labs. 2026. *tree: Real-time tree table*. Python documentation, version 0.40.7. Accessed September 9, 2026. <https://deephaven.io/core/0.40.7/docs/reference/table-operations/create/tree/>.
- [7] Deephaven Data Labs. 2026. *Execution context*. Official documentation, custom update graphs. Accessed September 9, 2026. <https://deephaven.io/core/docs/conceptual/execution-context/>.
- [8] Emil Dumitrescu, Alain Girault, Hervé Marchand, and Éric Ruten. 2007. *Optimal discrete controller synthesis for the modeling of fault-tolerant distributed systems*. INRIA Research Report RR-6137. https://www.researchgate.net/publication/250079523_Optimal_discrete_controller_synthesis_for_modeling_fault-tolerant_distributed_systems.
- [9] Shie Mannor, Ofir Mebel, and Huan Xu. 2012. Lightning Does Not Strike Twice: Robust MDPs with Coupled Uncertainty. In *ICML*. <https://icml.cc/2012/papers/215.pdf>.
- [10] Daniel Neider, Alexander Weinert, and Martin Zimmermann. 2020. Synthesizing optimally resilient controllers. *Acta Informatica* 57, 195–221. <https://doi.org/10.1007/s00236-019-00345-7>.
- [11] Alexander Weinert. 2019. *Optimality and Resilience in Parity Games*. PhD thesis, Saarland University. Section 6.2. https://finkbeiner.groups.cispa.de/_assets/paper.C89PMq4G.pdf.
- [12] Hyung-Chan An and Hoesook Yang. 2021. Fully Adaptive Stochastic Handling of Soft-Errors in Real-Time Systems. *IEEE Access* 9, 155058–155071. <https://doi.org/10.1109/ACCESS.2021.3128397>.
- [13] Awi Federgruen and Henri Groenevelt. 1986. The Greedy Procedure for Resource Allocation Problems: Necessary and Sufficient Conditions for Optimality. *Operations Research*. https://business.columbia.edu/sites/default/files-efs/pubfiles/4071/federgruen_greedy_procedure.pdf.
- [14] Dimitris Bertsimas and Melvyn Sim. 2004. The Price of Robustness. *Operations Research* 52, 1, 35–53. <https://www.mit.edu/~dberstsim/papers/Robust%20Optimization/The%20price%20of%20Robustness.pdf>.
- [15] Marek Chrobak, Mathilde Hurand, and Jiří Sgall. 2009. Algorithms for Testing Fault-Tolerance of Sequenced Jobs. *Journal of Scheduling*. Author version. <https://iuk.mff.cuni.cz/~sgall/ps/faults.pdf>.
- [16] Phebe Vayanos, Angelos Georgiou, and Han Yu. 2026. Robust Optimization with Decision-Dependent Information Discovery. *Management Science* 72, 2, 1509–1528. <https://doi.org/10.1287/mnsc.2021.00160>.
- [17] Eva Ley, Rebecca Marx, Maximilian Merkert, Tim Niemann, and Sebastian Stiller. 2026. Decision-dependent robust optimization with elementwise controllable uncertainty sets. *OR Spectrum*. <https://doi.org/10.1007/s00291-026-00856-z>.
- [18] PRISM-games developers. 2026. PRISM-games. Official tool documentation. Version 3.2.4 used in the experiments. <https://www.prismmodelchecker.org/games/>.
- [19] Pavol Černý, Krishnendu Chatterjee, Thomas A. Henzinger, Arjun Radhakrishna, and Rohit Singh. 2011. Quantitative Synthesis for Concurrent Programs. In *CAV*, 243–259. https://doi.org/10.1007/978-3-642-22110-1_20. Author publication page and complexity erratum: <https://www.microsoft.com/en-us/research/publication/quantitative-synthesis-for-concurrent-programs/>.
- [20] Pavol Černý, Edmund M. Clarke, Thomas A. Henzinger, Arjun Radhakrishna, Leonid Ryzhyk, Roopsha Samanta, and Thorsten Tarrach. 2015. Optimizing Solution Quality in Synchronization Synthesis. *arXiv preprint arXiv:1511.07163v1*.

- <https://arxiv.org/abs/1511.07163>.
- [21] Namratha Gangamreddypalli, Constantin Enea, and Shaz Qadeer. 2026. Reduction for Structured Concurrent Programs. *arXiv preprint* arXiv:2601.13341. <https://arxiv.org/abs/2601.13341>.
 - [22] Viacheslav Izosimov, Paul Pop, Petru Eles, and Zebo Peng. 2012. Scheduling and Optimization of Fault-Tolerant Embedded Systems with Transparency/Performance Trade-Offs. *ACM Transactions on Embedded Computing Systems* 11, 3, Article 61, 35 pages. <https://doi.org/10.1145/2345770.2345773>.
 - [23] Bingham Wu, Wei Bao, Dong Yuan, and Bing Zhou. 2022. Competitive Analysis for Multi-Commodity Ski-Rental Problem. In *IJCAI*, 4672–4678. <https://doi.org/10.24963/ijcai.2022/648>.
 - [24] Oracle. 2021. ConcurrentHashMap, Java SE 17 API Specification. Accessed September 7, 2026. <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/concurrent/ConcurrentHashMap.html>.
 - [25] Oracle. 2021. ConcurrentMap, Java SE 17 API Specification. Accessed September 7, 2026. <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/concurrent/ConcurrentMap.html>.
 - [26] OpenJDK. 2026. ConcurrentHashMap.java, jdk 17 u source, commit 982d53c227943a511e8d61694feced764b71a6c1. Accessed September 7, 2026. <https://github.com/openjdk/jdk17u/blob/982d53c227943a511e8d61694feced764b71a6c1/src/java.base/share/classes/java/util/concurrent/ConcurrentHashMap.java>.
 - [27] Ziwei Li, Bo Sun, Zhiqiu Zhang, Mohammad Hajiesmaili, Bingham Wu, Lin F. Yang, and Yang Gao. 2025. Combinatorial Ski Rental Problem: Robust and Learning-Augmented Algorithms. In *NeurIPS*. https://papers.nips.cc/paper_files/paper/2025/hash/ae3e173398d6e43fea63cbcc16fbaa98-Abstract-Conference.html.
 - [28] Krzysztof Burdzy, Weining Kang, and Kavita Ramanan. 2009. The Skorokhod problem in a time-dependent interval. *Stochastic Processes and their Applications* 119, 2, 428–452. <https://doi.org/10.1016/j.spa.2008.03.001>.
 - [29] Eugene L. Lawler. 1973. Optimal Sequencing of a Single Machine Subject to Precedence Constraints. *Management Science* 19, 5, 544–546. <https://doi.org/10.1287/mnsc.19.5.544>.
 - [30] Laurent Perron, Frédéric Didier, and Steven Gay. 2023. The CP-SAT-LP Solver (Invited Talk). In *CP, LIPIcs* 280, 3:1–3:2. <https://doi.org/10.4230/LIPIcs.CP.2023.3>.
 - [31] Krzysztof Postek, Ward Romeijnnders, and Wolfram Wiesemann. 2025. Technical Note: Multi-Stage Robust Mixed-Integer Programming. *Operations Research* 73, 6, 3345–3358. <https://doi.org/10.1287/opre.2023.0520>. [Online appendix; author code](#).
 - [32] Dimitris Bertsimas and Melvyn Sim. 2003. Robust discrete optimization and network flows. *Mathematical Programming* 98, 49–71. <https://doi.org/10.1007/s10107-003-0396-4>.
 - [33] Matthew Bold and Marc Goerigk. 2021. A compact reformulation of the two-stage robust resource-constrained project scheduling problem. *Computers & Operations Research*. <https://doi.org/10.1016/j.cor.2021.105232>. [Accepted manuscript](#).
 - [34] Richard M. Karp. 1972. Reducibility Among Combinatorial Problems. In *Complexity of Computer Computations*, 85–103. https://doi.org/10.1007/978-1-4684-2001-2_9. [Primary paper](#).
 - [35] Anonymous Authors. 2026. *Compiling Optimal Retry Policies for Dependent Transformations: Research Artifact*. Version 5 with publication supplement, commit e0e0110. <https://github.com/research-artifact-archive/research-artifact/tree/e0e011023c0591bbb422390ad8d29785ad0023f>.
 - [36] Anonymous Authors. 2026. *Retry Synthesis with Bounded Calls and Protected Work: Research Artifact*. Version 11 suffix supplement, commit 7a05b61. <https://github.com/research-artifact-archive/research-artifact/tree/7a05b6163370f02893e8275fcb3ae2e24609ae63>.